

• Лабораторная работа по ML №1

Жуховицкий А. Д. (М8О-303Б-23)

Установим все необходимые зависимости

```
import numpy as np
!uv sync --frozen --no-install-project --no-dev --no-cache
```

Audited **118 packages** in 1ms

Загрузим датасет

```
from kagglehub import dataset_download

DATASET_PATH = dataset_download("vikrishnan/boston-house-prices") + "/housing.csv"
COLUMN_NAMES = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'L']
```

Проверим поступившие данные

```
import pandas as pd
df = pd.read_csv(DATASET_PATH, header=None, delimiter=r"\s+", names=COLUMN_NAMES)
df
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	L
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273.0	21.0	391.99	
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273.0	21.0	396.90	
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273.0	21.0	396.90	
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273.0	21.0	393.45	
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273.0	21.0	396.90	

506 rows x 14 columns

Рассмотрим типы и основные данные для колонок

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype  
---  -
0    CRIM        506 non-null    float64
1    ZN          506 non-null    float64
2    INDUS       506 non-null    float64
3    CHAS        506 non-null    int64   
4    NOX         506 non-null    float64
5    RM          506 non-null    float64
6    AGE         506 non-null    float64
7    DIS         506 non-null    float64
8    RAD         506 non-null    int64   
9    TAX         506 non-null    float64
10   PTRATIO     506 non-null    float64
11   B           506 non-null    float64
12   LSTAT       506 non-null    float64
13   MEDV        506 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.5 KB
```

Пропусков значений не обнаружено, можно работать с данными без дополнительной обработки

```
df.describe()
```

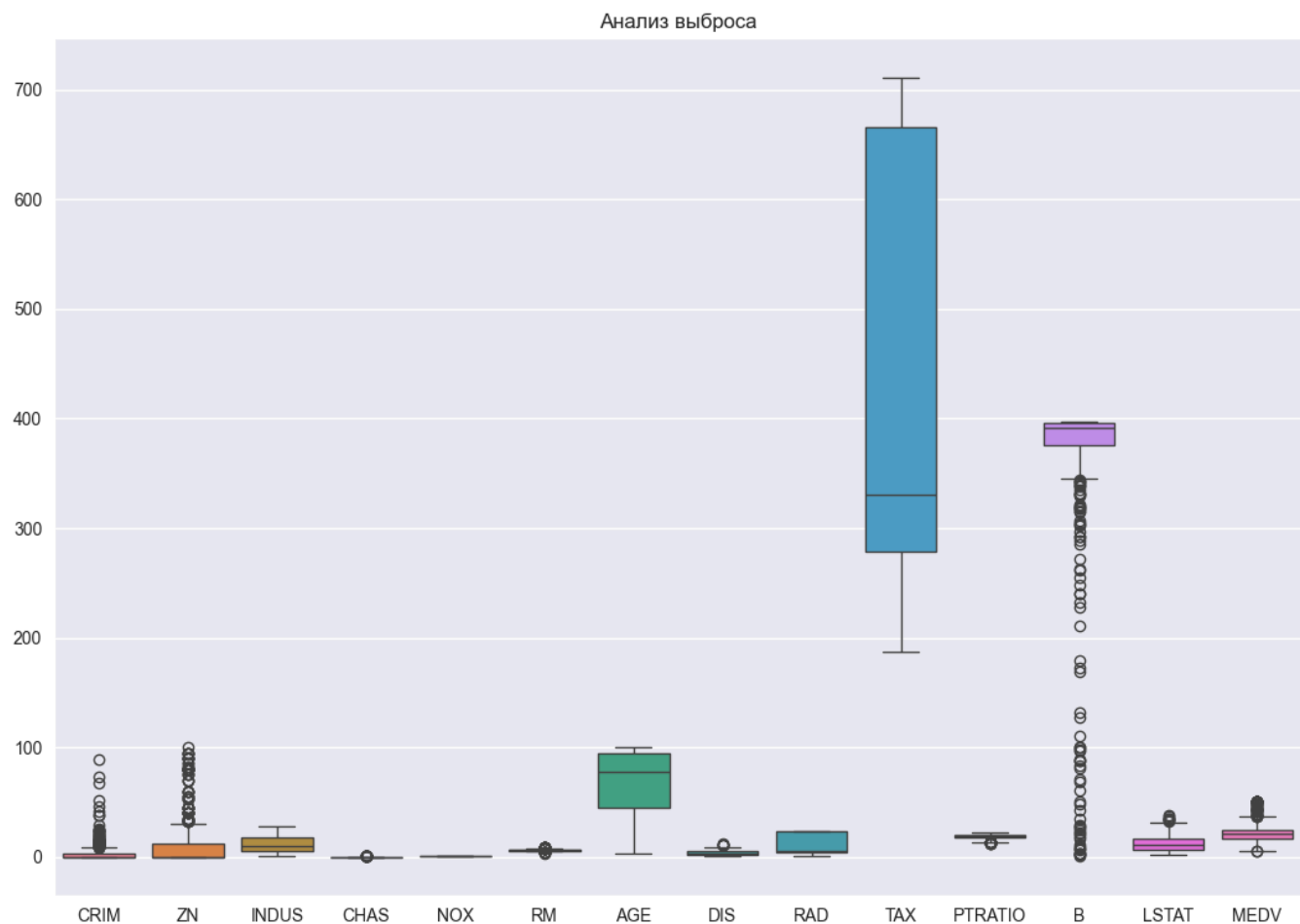
	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000

Проанализируем выбросы данных, для этого визуализируем данные

```
# Импортируем библиотеки для графиков
import seaborn as sns
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(13, 9))
sns.boxplot(df)
```

```
plt.title("Анализ выброса")
plt.show()
```



Можно заметить выбросы данных по точкам выделенным на плоте.

Проанализируем корреляцию между колонками. Визуализируем матрицу корреляции, используя heatmap.

```
plt.figure(figsize=(13, 9))
sns.heatmap(df.corr(), annot=True)
plt.title("Corr mtx")
plt.show()
```



При анализе корреляций можно смотреть на коэффициенты, которые по модулю превышают 0.7, чтобы с меньшей вероятностью наткнуться на ложные корреляции и отфильтровать их, однако можно рассмотреть и частичные корреляции. Построим хитмапу корреляций для пар колонок без зависимости от внешних факторов.

```
from pingouin import partial_corr
from itertools import combinations

partial_corr_mtx = df.corr()
for x, y in combinations(COLUMN_NAMES, 2):
    others = [c for c in df.columns if c not in (x, y)]
    res = partial_corr(data=df, x=x, y=y, covar=others)
    partial_corr_mtx[x][y] = partial_corr_mtx[y][x] = res['r'].iloc[0]

plt.figure(figsize=(13, 9))
sns.heatmap(partial_corr_mtx, annot=True)
plt.show()
```



Определим каждую колонку:

- CRIM - уровень преступности в городе
- ZN - доля жилой зоны более 25,000 квадратных футов
- INDUS - доля акров не ретейлерского бизнеса в городе
- CHAS - 1 если имеет общую границу с рекой, 0 если нет
- NOX - концентрация оксидов азота (частей на 10 млн)
- RM - среднее количество комнат в жилище
- AGE - доля занятой земли застроенной до 1940 года
- DIS - взвешанное расстояние от 5 бостонских центров трудоустройства
- RAD - индекс близости к радиальным шоссе
- TAX - налог на полную стоимость земли на 10 000 долларов
- PTRATIO - доля количества студентов к количеству учителей в городе
- B - $1000 * (B_k - 0,63) ^ 2$, где B_k - доля чернокожих в городе
- LSTAT - % населения в низком статусе
- MEDV - медианная стоимость занятой земли в 1000 долларов

На матрице частичных корреляций действительно заметна логика между корреляцией конкретных признаков.

Попробуем предсказывать MEDV, используя все остальные данные. Разделим данные на цель (target) и параметры.

```
target = df["MEDV"]
target
```

```
0      24.0
1      21.6
2      34.7
3      33.4
4      36.2
...
501     22.4
502     20.6
503     23.9
504     22.0
505     11.9
Name: MEDV, Length: 506, dtype: float64
```

```
params = df.drop("MEDV", axis=1)
params
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	L
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273.0	21.0	391.99	
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273.0	21.0	396.90	
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273.0	21.0	396.90	
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273.0	21.0	393.45	
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273.0	21.0	396.90	

506 rows × 13 columns

Поделим данные на train и test, будем использовать sklearn для работы с данными и обучения.

```
from sklearn.model_selection import train_test_split

params_train, params_test, target_train, target_test = train_test_split(params, target)
```

Для удобной работы с последовательной обработкой scaler'ом и инструментами sklearn используем Pipeline'ы и составим последовательности обработки с разными вариантами

моделей.

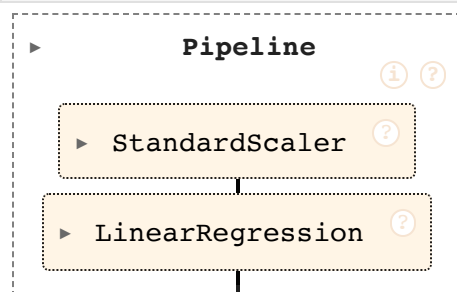
```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

После обучения моделей снимем метрики и проверим какой вариант покажет себя оптимально.

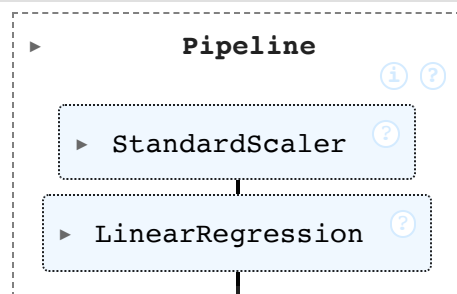
```
# Импортируем готовые методы для вычисления метрик
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, root_mse
```

- Классическая линейная регрессия

```
from sklearn.linear_model import LinearRegression
linear_regression_pipeline = Pipeline([("Scaler", StandardScaler()), ("LinearRegression", LinearRegression())])
linear_regression_pipeline.fit(params_train, target_train)
```



```
linear_regression_pipeline.fit(params_train, target_train)
```



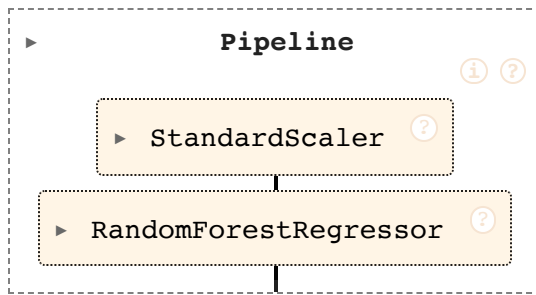
После обучения получим каждую модель и predicted данные.

```
model_1 : LinearRegression = linear_regression_pipeline.named_steps["LinearRegression"]
target_pred_1 = linear_regression_pipeline.predict(params_test)
```

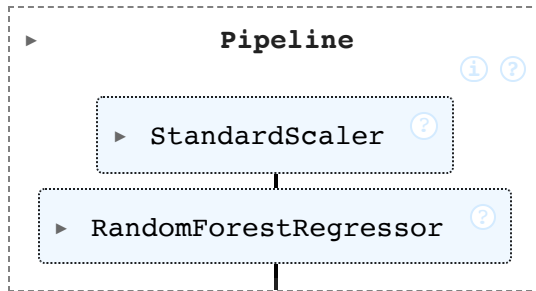
- Random forest

```
from sklearn.ensemble import RandomForestRegressor

random_forest_regressor_pipeline = Pipeline([("Scaler", StandardScaler()), ("RandomForestRegressor", RandomForestRegressor())])
random_forest_regressor_pipeline.fit(params_train, target_train)
```



```
random_forest_regressor_pipeline.fit(params_train, target_train)
```

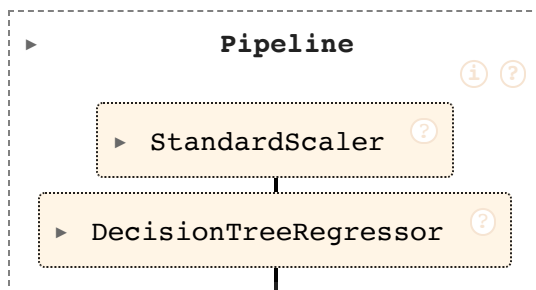


```
model_2 : RandomForestRegressor = random_forest_regressor_pipeline.named_steps["RandomForestRegressor"]
target_pred_2 = linear_regression_pipeline.predict(params_test)
```

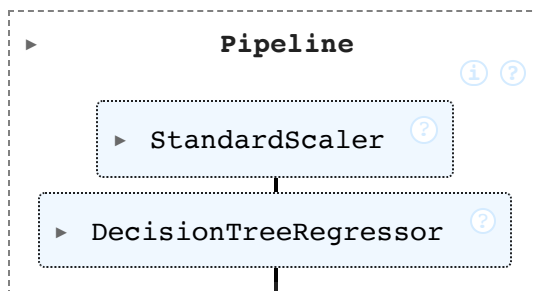
- Decision tree (дерево решений)

```
from sklearn.tree import DecisionTreeRegressor
```

```
decision_tree_regressor_pipeline = Pipeline([("Scaler", StandardScaler()), ("DecisionTreeRegressor", DecisionTreeRegressor())])
```



```
decision_tree_regressor_pipeline.fit(params_train, target_train)
```



```
model_3 : DecisionTreeRegressor = decision_tree_regressor_pipeline.named_steps["DecisionTreeRegressor"]
target_pred_3 = decision_tree_regressor_pipeline.predict(params_test)
```


Проанализируем результаты

Сравним MAE, будем искать минимальные

```
# Linear regression
MAE1 = mean_absolute_error(target_test, target_pred_1)
MAE1
```

3.436331338991636

```
# Random forest
MAE2 = mean_absolute_error(target_test, target_pred_2)
MAE2
```

3.436331338991636

```
# Decision tree
MAE3 = mean_absolute_error(target_test, target_pred_3)
MAE3
```

2.747244094488189

Сравним MSE, будем искать минимальные

```
# Linear regression
MSE1 = mean_squared_error(target_test, target_pred_1)
MSE1
```

19.93096288994256

```
# Random forest
MSE2 = mean_squared_error(target_test, target_pred_2)
MSE2
```

19.93096288994256

```
# Decision tree
MSE3 = mean_squared_error(target_test, target_pred_3)
MSE3
```

14.183385826771653

Сравним r^2 , будем искать максимальный

```
# Linear regression
R2_1 = r2_score(target_test, target_pred_1)
R2_1
```

0.7650391351458242

```
# Random forest
R2_2 = r2_score(target_test, target_pred_2)
```

```
R2_2
```

```
0.7650391351458242
```

```
# Decision tree
```

```
R2_3 = r2_score(target_test, target_pred_3)
```

```
R2_3
```

```
0.8327958052593448
```

- Decision Tree показывает наилучший результат среди нескольких испытаний!

Рассмотрим влияние каждого параметра на итоговый результат для каждой модели, проанализируем веса.

```
pd.DataFrame({"Params": params_train.columns, "Weights": model_1.coef_})
```

	Params	Weights
0	CRIM	-1.182803
1	ZN	1.119359
2	INDUS	0.094376
3	CHAS	0.524098
4	NOX	-2.137763
5	RM	2.182827
6	AGE	0.081960
7	DIS	-3.259755
8	RAD	3.272377
9	TAX	-2.545151
10	PTRATIO	-2.270262
11	B	0.854109
12	LSTAT	-3.963336

```
pd.DataFrame({"Params": params_train.columns, "Weights": model_2.feature_importances_})
```

	Params	Weights
0	CRIM	0.048494
1	ZN	0.000978
2	INDUS	0.007079
3	CHAS	0.001194
4	NOX	0.023619
5	RM	0.406548
6	AGE	0.014998
7	DIS	0.065305
8	RAD	0.003641
9	TAX	0.012908
10	PTRATIO	0.020931
11	B	0.014696
12	LSTAT	0.379611

```
pd.DataFrame({"Params": params_train.columns, "Weights": model_3.feature_importances_
```

	Params	Weights
0	CRIM	0.038219
1	ZN	0.002267
2	INDUS	0.002934
3	CHAS	0.000000
4	NOX	0.044174
5	RM	0.575556
6	AGE	0.007627
7	DIS	0.078297
8	RAD	0.002501
9	TAX	0.025057
10	PTRATIO	0.014440
11	B	0.009212
12	LSTAT	0.199714